Integración de pagos en cripto con PagArte

Guía técnica de implementación

Artemis Money · PagArte Checkout

Preparado por **Aurigital** 22 de octubre de 2026

Esta guía documenta cómo integrar el checkout de PagArte para cobrar en **cripto, SINPE Móvil y Artemis App** desde un sitio propio.

Todo lo que contiene está verificado contra la API real, incluyendo un cobro en producción con dinero real.

Contenido

1. Qué es PagArte y en qué se diferencia de otras pasarelas
2. Preparar la cuenta
3. Montos y moneda
4. Crear la sesión en el servidor
5. Idempotencia y sesiones activas
6. Mostrar el checkout en el navegador
7. Confirmar el pago con webhooks firmados
8. Códigos de error
9. Probar en sandbox
10. Pasar a producción
11. Liquidación

Apéndices: persistencia del estado y referencia rápida de la API.

Manual de integración — PagArte (Artemis Money)

Guía para integrar el checkout de **PagArte** en un ecommerce y cobrar en **cripto, SINPE Móvil y Artemis App**.

Está escrita para una persona desarrolladora que no conoce PagArte. Los ejemplos son de Next.js, pero el flujo y las reglas aplican a cualquier stack (hay ejemplos oficiales en Node, Python, PHP y Go en la documentación de PagArte).

> **Documentación oficial:** https://pagarte.artemis.money/docs/es

---

Estado de esta guía

Todo lo que sigue está **verificado contra la API real**, en sandbox y en producción, el 22 de octubre de 2026:

Verificación	Resultado
Crear sesión, idempotencia, rechazo de moneda y origen	✅
Checkout embebido con el SDK	✅
Pago exacto → compra completada	✅
Pago parcial → la compra **no** se completa	✅
Evento duplicado → no se reprocesa	✅
Firma inválida → `400`	✅
Cobro real en Live	✅ ₡1 000 → 2.222195 USDT liquidados en Tron

> Hubo un bug en el checkout embebido de PagArte (`frame-ancestors 'none'`) que lo hacía imposible de mostrar. Se reportó y **Artemis lo corrigió el 21/10/2026**. Si volviera a aparecer un iframe en blanco, el detalle está en `PAGARTE_IFRAME_ISSUE.md` .

1. Qué es PagArte y en qué se diferencia

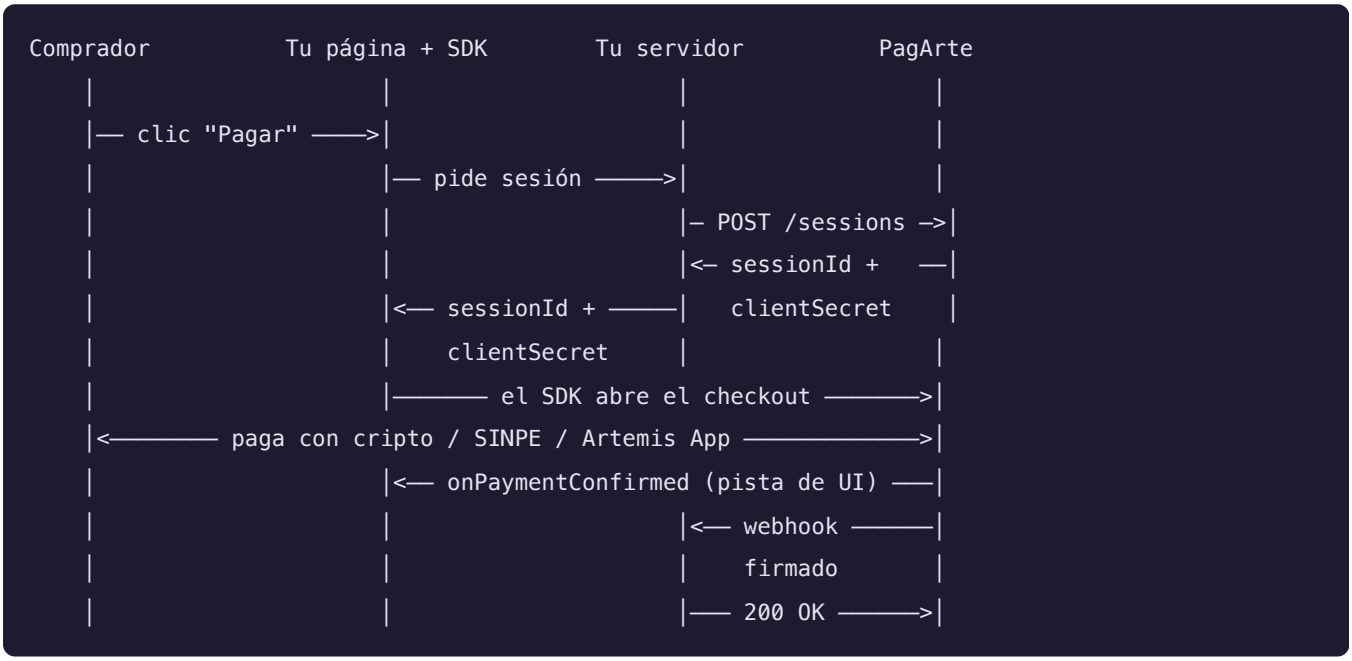
Si venís de Stripe, ONVO o TiloPay, hay una diferencia de fondo que conviene entender antes de escribir código:

PagArte no genera un link de pago hospedado.

	Pasarela con link (ONVO, TiloPay)	PagArte
Qué devuelve la API	Una URL a la que redirigís	`sessionId` + `clientSecret`
Dónde paga el cliente	En el sitio de la pasarela	En **tu propia página**, en un iframe
Cómo confirmás el pago	Redirect de vuelta	**Webhook firmado**
Requisito extra	—	Tu dominio debe estar **autorizado** en el panel

Esto implica que necesitás **dos piezas**:

1. **Un endpoint en tu servidor** que crea la sesión (ahí vive la llave secreta).
2. **Una página tuya** que monta el SDK y muestra el checkout.



> ⚠️ **La regla más importante:** `onPaymentConfirmed` del navegador es **solo una pista visual**. Nunca completes una compra con eso. La única confirmación válida es el **webhook firmado** (sección 7).

---

2. Preparar la cuenta

Todo se hace en el panel de Artemis: **Ventas Online**. Hay dos modos independientes, **Sandbox** y **Live**, cada uno con su propio widget, llaves y webhook.

2.1 Activar y autorizar tu sitio

En **Ventas Online** → **Configuración**:

1. **Recibir pagos:** activá el interruptor.
2. **Sitios autorizados:** agregá cada origen donde vas a mostrar el checkout.
3. Debe ser el **origen exacto**: `https://tutienda.com`, sin ruta ni barra final.

4. En sandbox podés usar `http://localhost:3000`.
5. Si el origen no coincide exactamente, la API responde `403 CHECKOUT_WIDGET_BLOCKED`.
6. **Métodos de pago:** elegí cuáles aceptar.
7. *SINPE Móvil* requiere un teléfono registrado para recibir.
8. *Cripto* y *Artemis App* requieren configurar cómo recibís el dinero.
9. **Guardá el ID del widget** — lo vas a necesitar como `PAGARTE_WIDGET_ID`.

2.2 Crear la llave de API

En **Llaves de API**, creá una con un nombre reconocible.

> 🔑 **La llave se muestra UNA SOLA VEZ.** Guardala en el gestor de secretos o en las variables de entorno del servidor. Si se filtra, revocala y creá otra: las integraciones que la usen dejan de funcionar de inmediato.

La llave define el modo: `pgart_test_…` es sandbox, `pgart_live_…` es Live. La URL de la API es la misma para ambos.

2.3 Configurar el webhook

En **Notificaciones (webhook)**, poné la URL pública que va a recibir los eventos y marcá los tres: `checkout.payment.confirmed`, `checkout.cancelled`, `checkout.expired`.

Requisitos de la URL:

- En Live debe ser **HTTPS**, sin puerto ni query string.
- **No se envían webhooks a `localhost`.** Para desarrollo usá un túnel:

``` `bash ngrok http 3000 # → https://abc123.ngrok-free.app/api/pagarte/webhook` ```

2.4 Variables de entorno

```
PAGARTE_API_URL=https://api.artemis.money
NEXT_PUBLIC_PAGARTE_CHECKOUT_ORIGIN=https://checkout.pagarte.artemis.money
PAGARTE_SECRET_KEY=pgart_test_...      # solo servidor, nunca al navegador
PAGARTE_WIDGET_ID=...                  # distinto en sandbox y en Live
STORE_ORIGIN=https://tutienda.com      # debe ser un sitio autorizado
```

---

3. Montos y moneda

PagArte cobra únicamente en colones (CRC). Cualquier otra moneda responde `422 CHECKOUT_CURRENCY_NOT_ENABLED`.

Los montos van en **unidades menores, como string**:
```

Monto	`amountMinor`
₡10.00	`"1000"`
₡1 500.50	`"150050"`
₡25 000	`"2500000"`

```
const amountMinor = String(Math.round(montoEnColones * 100))
```

---

4. Crear la sesión (servidor)

```
// Corre en TU servidor: la llave secreta nunca llega al navegador.
export async function createCheckoutSession(params: {
  merchantOrderId: string
  amountMinor: string
  idempotencyKey: string
}) {
  const response = await fetch(`${process.env.PAGARTE_API_URL}/v1/checkout/sessions`, {
    method: 'POST',
    headers: {
      Authorization: `Bearer ${process.env.PAGARTE_SECRET_KEY}`,
      'Idempotency-Key': params.idempotencyKey,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      widgetId: process.env.PAGARTE_WIDGET_ID,
      merchantOrderId: params.merchantOrderId,
      amount: { currency: 'CRC', amountMinor: params.amountMinor },
      embeddingOrigin: process.env.STORE_ORIGIN,
    }),
  })

  const body = await response.json()
  if (!response.ok) {
    throw new Error(`PagArte ${response.status} ${body?.error?.code}`)
  }

  // 201 = sesión nueva. 200 = la misma sesión (Idempotency-Key repetida).
  return {
    sessionId: body.sessionId,
    clientSecret: body.clientSecret,
    expiresAt: body.expiresAt,
  }
}
```

Campos

Campo	Qué es
`widgetId`	El ID del widget, **del mismo modo que la llave**
`merchantOrderId`	Tu referencia de la compra. Única en tu empresa y modo, **siempre la misma para esa compra**. Letras, dígitos y `. _ : -`
`amount`	`{ currency: 'CRC', amountMinor: '...' }`
`embeddingOrigin`	El sitio que mostrará el checkout. Debe estar autorizado

Exponé tu propio endpoint

Tu página nunca llama a PagArte directamente. Llama a un endpoint tuyo, que valida que ese usuario puede pagar esa compra y devuelve **solo los tres campos**:

```
// POST /api/purchases/[id]/checkout
export async function POST(_req: NextRequest, { params }: { params: { id: string } }) {
  const purchase = await getPurchaseForUser(params.id, await getCurrentUser())
  if (!purchase) return new NextResponse(null, { status: 404 })

  const session = await createCheckoutSession({
    merchantOrderId: purchase.id,
    amountMinor: purchase.amountMinor,
    idempotencyKey: purchase.checkoutAttemptKey,
  })

  // Solo sessionId, clientSecret y expiresAt. Nada más.
  return NextResponse.json(session)
}
```

---

5. Idempotencia y una sesión activa por compra

`Idempotency-Key`

Caracteres permitidos: `A-Z a-z 0-9 _ -`.

- Misma clave + mismo cuerpo → **la misma sesión** (HTTP 200).
- Misma clave + cuerpo distinto → `409 CHECKOUT_IDEMPOTENCY_CONFLICT`.

Guardá la clave del intento junto con la compra y reusala mientras la sesión siga viva. Eso es lo que permite que el comprador recargue la página y continúe el mismo pago. Generá una clave nueva solo después de que la sesión anterior se cancele o venza.

Una sola sesión activa

Respuesta	Cuándo	Qué hacer
`409 CHECKOUT_ORDER_ACTIVE`	La compra ya tiene una sesión activa	Reusá la clave del intento activo, o cancelalo y creá otro
`409 CHECKOUT_ORDER_AMOUNT_CONFLICT`	Esa referencia ya se usó con otro monto o moneda	Si cambió el carrito, usá una **referencia nueva**
`409 CHECKOUT_ORDER_BLOCKED`	Ya está pagada o tiene un pago en revisión	**No cobres de nuevo.** Consultá la sesión anterior

Cancelar

```
curl -X POST "$PAGARTE_API_URL/v1/checkout/sessions/$SESSION_ID/cancel" \
  -H "Authorization: Bearer $PAGARTE_SECRET_KEY" \
  -H "Idempotency-Key: cancel-pedido-1001-intento-1" \
  -H "Content-Type: application/json" \
  -d '{}'
```

---

6. Mostrar el checkout (navegador)

El SDK es un módulo JavaScript que se carga desde el origen del checkout.

```
'use client'

const SDK_URL = `${process.env.NEXT_PUBLIC_PAGARTE_CHECKOUT_ORIGIN}/sdk/v1/checkout.js`

export function PagArteCheckout({ purchaseId }: { purchaseId: string }) {
  const containerRef = useRef<HTMLDivElement>(null)
  const checkoutRef = useRef<CheckoutHandle | null>(null)

  useEffect(() => {
    let destroyed = false

    async function start() {
      // El comentario evita que el bundler intente empaquetar un módulo remoto.
      const { mount } = await import(/* webpackIgnore: true */ SDK_URL)
      if (destroyed || !containerRef.current) return

      checkoutRef.current = mount({
        container: containerRef.current,

        fetchSession: async () => {
          const r = await fetch(`/api/purchases/${purchaseId}/checkout`, { method: 'POST' })
          if (!r.ok) throw new Error('No pudimos iniciar el pago')
          return r.json()
        },

        onStatus: ({ status }) => { /* mensajes de UI */ },

        // Pista visual. La compra la completa el webhook.
        onPaymentConfirmed: () => router.push(`/compras/${purchaseId}/procesando`),

        onClose: async ({ status, cancellationPending }) => {
          if (cancellationPending) {
            await fetch(`/api/purchases/${purchaseId}/checkout/cancel`, { method: 'POST' })
          }
        },

        onError: ({ code, retryable }) => { /* retryable → reintenta solo */ },
      })
    }

    void start()

    return () => {
      destroyed = true
      // destroy(), NO close(): salir de la página no debe cancelar el pago.
```

```
      checkoutRef.current?.destroy()
    }
  }, [purchaseId])

  return <div ref={containerRef} />
}
```

> En React, Next.js, Angular y Vue **el comentario dentro de `import()` es obligatorio** (`webpackIgnore` o `@vite-ignore` según el bundler). Sin él, el build falla al intentar resolver la URL remota.

`destroy()` vs `close()`

Método	Qué hace	Cuándo
`destroy()`	Quita el iframe **sin cancelar**	El comprador navega a otro lado. Si vuelve, sigue la misma sesión
`close()`	**Cancela** el pago y quita el iframe	El comprador aprieta "cancelar"

Estados (`onStatus`)

Estado	Significado
`CREATED`	Sesión creada; aún no eligió método
`PREPARING`	Preparando las instrucciones
`AWAITING_PAYMENT`	Instrucciones visibles; esperando el pago
`CONFIRMING`	Llegó un pago, pero el monto aún no está completo
`CONFIRMED`	Pago confirmado
`CANCELLED` / `EXPIRED` / `FAILED`	Terminales

Content Security Policy

Si usás CSP, permitÍ el origen del checkout:

```
Content-Security-Policy: script-src 'self' https://checkout.pagarte.artemis.money;
                         frame-src https://checkout.pagarte.artemis.money
```

---

7. Confirmar el pago: el webhook

Esta es la parte que no se puede hacer a medias. Una firma mal verificada permite que cualquiera marque compras como pagadas.

La firma

Cada webhook trae el header:

```
Pagarte-Signature: t=1760553730,kid=Xr4Tq8Lm2Vn6Pz1W,ed25519=<base64url>
```

La firma es **Ed25519 sobre los bytes de** `<t>.<cuerpo crudo>`.

> ⚠️ Verificá el **cuerpo tal cual llegó**. Si lo parseás a JSON y lo volvés a serializar, la firma no va a coincidir. En Next.js: `await request.arrayBuffer()`, nunca `await request.json()` antes de verificar.

Implementación

```
import { createPublicKey, verify, type KeyObject } from 'node:crypto'

const SIGNATURE =
  /^t=([0-9]{1,12}),kid=([A-Za-z0-9_-]{8,128}),ed25519=([A-Za-z0-9_-]{86})$/
const TOLERANCE_SECONDS = 300
const MIN_REFRESH_MS = 60_000

export class PagarteWebhookKeys {
  private keys = new Map<string, KeyObject>()
  private lastRefresh = 0
  constructor(private readonly apiUrl: string) {}

  async get(kid: string): Promise<KeyObject | null> {
    // Un kid desconocido significa rotación de llaves: recargá (con límite).
    if (!this.keys.has(kid) && Date.now() - this.lastRefresh > MIN_REFRESH_MS) {
      await this.refresh()
    }
    return this.keys.get(kid) ?? null
  }

  private async refresh() {
    this.lastRefresh = Date.now()
    const r = await fetch(`${this.apiUrl}/v1/checkout/webhook-keys`)
    if (!r.ok) return
    const { keys } = await r.json()
    for (const k of keys) {
      if (k.algorithm === 'Ed25519') this.keys.set(k.kid, createPublicKey(k.publicKey))
    }
  }
}

export async function verifyPagarteWebhook(
  rawBody: Buffer,
  header: string | undefined,
  keys: PagarteWebhookKeys,
  now = Date.now(),
): Promise<boolean> {
  const m = SIGNATURE.exec(header ?? '')
  if (!m) return false
  const [, t, kid, sig] = m

  // Rechaza reenvíos viejos.
  if (Math.abs(Math.floor(now / 1000) - Number(t)) > TOLERANCE_SECONDS) return false
```

```
  const key = await keys.get(kid)
  if (!key) return false

  return verify(
    null,
    Buffer.concat([Buffer.from(`${t}.`), rawBody]),
    key,
    Buffer.from(sig, 'base64url'),
  )
}
```

Los chequeos obligatorios

Una firma válida prueba que el evento **viene de PagArte**, no que corresponde a tu compra. Antes de dar algo por pagado:

1. ✅ **Verificá la firma** sobre el cuerpo crudo.
2. ✅ **Rechazá si** `|ahora − t| > 300s` (anti-replay).
3. ✅ **Validá** `environment` (`TEST` vs `LIVE`) contra el modo de tu llave. Sin esto, un evento de sandbox podría completar una compra real.
4. ✅ **Buscá tu compra** por `data.merchantOrderId` y **compará el monto**.
5. ✅ **Procesá cada** `eventId` **una sola vez.** PagArte reenvía eventos.
6. ✅ **Respondé 2xx** (o `400` si la firma falla) para que deje de reintentar.

```
export const runtime = 'nodejs'       // node:crypto no corre en Edge
export const dynamic = 'force-dynamic'

export async function POST(request: NextRequest) {
  const rawBody = Buffer.from(await request.arrayBuffer())

  if (!(await verifyPagarteWebhook(rawBody, request.headers.get('Pagarte-Signature'), keys))) {
    return new NextResponse('Invalid signature', { status: 400 })
  }

  const event = JSON.parse(rawBody.toString('utf8'))

  const esperado = process.env.PAGARTE_SECRET_KEY!.startsWith('pgart_live_') ? 'LIVE' : 'TEST'
  if (event.environment !== esperado) return NextResponse.json({ ignored: true })

  if (await yaProcesado(event.eventId)) return NextResponse.json({ ok: true })

  const compra = await buscarCompra(event.data.merchantOrderId)
  if (!compra) return NextResponse.json({ ignored: true })
  if (event.data.amount?.amountMinor !== compra.amountMinor) {
    return NextResponse.json({ ignored: 'monto no coincide' })
  }

  if (event.type === 'checkout.payment.confirmed') await completarCompra(compra.id)
  await marcarProcesado(event.eventId)

  return NextResponse.json({ ok: true })
}
```

Consultar la sesión

Cuando el comprador vuelve antes que el webhook, o para reconciliar:

```
curl "$PAGARTE_API_URL/v1/checkout/sessions/$SESSION_ID" \
  -H "Authorization: Bearer $PAGARTE_SECRET_KEY"
```

---

8. Errores

Todos tienen el mismo formato:

```
{ "error": { "code": "...", "message": "...", "retryable": false, "requestId": "..." } }
```

Decidí por `code`, no por `message`. Si `retryable` es `true`, reintentá con demoras crecientes y la **misma** `Idempotency-Key`. Si contactás a soporte, incluí el `requestId`.

HTTP	Código	Qué hacer
400	`CHECKOUT_INVALID_REQUEST`	Revisá el cuerpo
400	`CHECKOUT_AMOUNT_OUT_OF_RANGE`	El monto es demasiado grande
400	`CHECKOUT_SIMULATION_SANDBOX_ONLY`	Simulaste con llave Live
401	`CHECKOUT_INVALID_CREDENTIAL`	Llave inválida o revocada
403	`CHECKOUT_WIDGET_BLOCKED`	Widget apagado o `embeddingOrigin` no autorizado
404	`CHECKOUT_WIDGET_NOT_FOUND`	El `widgetId` no existe en el modo de la llave
404	`CHECKOUT_SESSION_NOT_FOUND`	La sesión no existe para esta llave
409	`CHECKOUT_IDEMPOTENCY_CONFLICT`	Misma clave, cuerpo distinto
409	`CHECKOUT_ORDER_ACTIVE`	Ya hay una sesión activa
409	`CHECKOUT_ORDER_AMOUNT_CONFLICT`	Esa referencia ya se usó con otro monto
409	`CHECKOUT_ORDER_BLOCKED`	Ya pagada o en revisión
409	`CHECKOUT_PAYMENT_IN_PROGRESS`	Hay un pago en curso; no se puede cancelar
410	`CHECKOUT_SESSION_EXPIRED`	Venció. Creá otro intento
422	`CHECKOUT_CURRENCY_NOT_ENABLED`	Usá `CRC`
422	`CHECKOUT_MERCHANT_NOT_READY`	Falta configurar cómo recibís el dinero
429	`CHECKOUT_RATE_LIMITED`	Reintentá según `Retry-After`
429	`CHECKOUT_SESSION_QUOTA_EXCEEDED`	Cancelá las sesiones que no uses
503	`CHECKOUT_UNAVAILABLE`	Servicio no disponible. Reintentá
503	`CHECKOUT_PROVIDER_UNCERTAIN`	El proveedor no respondió. Reintentá igual

---

9. Probar en sandbox

Con una llave `pgart_test_…` **no se mueve dinero real**.

Desde **Ventas Online** → **Sandbox** podés simular:

Acción	Qué prueba
Pago exacto	El pago completo y `checkout.payment.confirmed`
Otro monto	Un pago parcial (la venta queda en `CONFIRMING`)
Moneda/destino equivocado	Pagos que no corresponden y se ignoran
Repetir el último pago	Un pago duplicado, que se ignora
Hacer que venza	La expiración y `checkout.expired`
Hacer fallar el próximo intento	Falla de preparación
Dejar la conversión pendiente	Liquidación en `PENDING`

Checklist antes de pasar a Live

- □ **Pago exacto:** tu webhook verifica la firma y completa la compra **una sola vez**.
- □ **El mismo evento dos veces** (reenvialo desde el panel): **no** se completa dos veces.
- □ **Pago parcial:** la compra **no** se completa.
- □ **Expiración y cancelación:** el carrito sigue disponible para otro intento.
- □ **El comprador recarga** durante el pago: vuelve a la **misma** sesión.
- □ **Firma manipulada:** tu servidor responde **400**.

---

10. Pasar a Live

Live se configura **aparte** de sandbox: otro widget, otras llaves, otra URL de notificación.

1. Completá la checklist de sandbox.
2. En **Ventas Online** (no Sandbox), agregá tu sitio real con `https://`.
3. Activá **Recibir pagos** y configurá **cómo recibís el dinero**:
4. *SINPE Móvil* → teléfono registrado.
5. *Cripto* → wallet, o cuenta en CRC si usás conversión automática.
6. Sin esto, la API responde `422 CHECKOUT_MERCHANT_NOT_READY`.
7. Creá la llave `pgart_live_…` y copiá el **nuevo** `PAGARTE_WIDGET_ID`.
8. Configurá la URL del webhook de producción (HTTPS, sin puerto).

Qué cambia

- **Los pagos son reales.** Cancelar una venta **no reembolsa** un pago que ya llegó.
- **No existen los endpoints de simulación** (`/v1/checkout/test/…`).
- Con conversión automática apagada, Live **solo ofrece las criptos para las que

tengas wallet**.

---

11. Liquidación

En **Configuración** → **Liquidación de órdenes cripto** elegís si Artemis convierte automáticamente a colones.

Conversión	Recibís
Activada	Colones al tipo de cambio del momento, menos comisiones
Desactivada	El cripto directo (USDT/USDC) en tu wallet

Con la conversión activada, **el monto neto puede diferir del monto de la venta** por tipo de cambio y comisiones. En `GET /v1/checkout/sessions/{id}`:

Campo	Qué es
`settlement.status`	`NOT_STARTED`, `PENDING`, `PROCESSING`, `COMPLETED`, `FAILED`, `NOT_APPLICABLE`
`settlement.currency`	`CRC`, `USDT`, `BTC`…
`settlement.netAmountMinor`	Neto en unidades menores
`settlement.decimals`	Decimales de esa moneda

Para obtener el monto: `netAmountMinor / 10^decimals`. Por ejemplo `123456` con `decimals: 8` es `0.00123456`.

> **La liquidación es independiente de la confirmación.** Completá la compra cuando el pago esté `CONFIRMED`, sin esperar a que la liquidación termine.

Ejemplo real

Un cobro de producción del 22/10/2026, con la conversión automática **desactivada**:

Cobrado	₡1 000,00
Método	USDT · Tron (TRC20)
Pago recibido	2,23 USDT
Liquidado al comercio	**2,222195 USDT**

La diferencia entre lo recibido y lo liquidado es la comisión. Con la conversión desactivada el USDT llega directo a la billetera de Artemis, sin tipo de cambio de por medio: el saldo aparece en **USDT (TRC20)**, no en colones.

Si en cambio la conversión está **activada**, se recibe CRC al tipo de cambio del momento, y ahí el neto puede diferir bastante del monto de la venta.

Referencia rápida

```
API              https://api.artemis.money
SDK              https://checkout.pagarte.artemis.money/sdk/v1/checkout.js
Tipos (.d.ts)    https://checkout.pagarte.artemis.money/sdk/v1/checkout.d.ts
Llaves webhook   GET {API}/v1/checkout/webhook-keys
Docs             https://pagarte.artemis.money/docs/es
```

Endpoint	Para qué
`POST /v1/checkout/sessions`	Crear la sesión
`GET /v1/checkout/sessions/{id}`	Estado y liquidación
`POST /v1/checkout/sessions/{id}/cancel`	Cancelar
`GET /v1/checkout/webhook-keys`	Llaves públicas para verificar firmas
`POST /v1/checkout/test/sessions/{id}/…`	Simulaciones (solo sandbox)

Implementación de referencia

Este repositorio tiene el flujo completo y funcionando:

Archivo	Qué hace
`lib/pagarte.ts`	Cliente de la API (crear, consultar, cancelar)
`lib/pagarte-webhook.ts`	Verificación Ed25519 y caché de llaves
`lib/purchases.ts`	Estado de compras e idempotencia de eventos
`app/api/pagarte/purchases/[id]/checkout/route.ts`	El `fetchSession` del SDK
`app/api/pagarte/webhook/route.ts`	Receptor del webhook con los 6 chequeos
`app/pay/[purchaseId]/page.tsx`	Página que monta el SDK
`middleware.ts`	Qué rutas son públicas para el comprador

Probar un pago en sandbox

En sandbox **no se mueve dinero real**. Los pagos se simulan desde el panel de Artemis; no se envía cripto desde una wallet.

La dirección de sandbox no es real

Al elegir cripto, el checkout muestra algo así:

```
PAGARTE_SANDBOX_TRX_ddc3f236-a4c9-4ede-9f72-1b2cddb5efa8
```

Eso **no es una dirección de Tron** (las reales empiezan con `T` y tienen 34 caracteres). Es un marcador de sandbox que no existe en la blockchain, por eso cualquier wallet lo rechaza como inválido. Es el comportamiento esperado: el banner amarillo del checkout lo advierte con *"Modo de prueba · No envíes dinero real"*.

Cómo simular el pago

1. Generá un cobro en https://pagarte.aurigital.com y abrí el link.
2. Elegí **Cripto** y esperá a que aparezcan el QR y la dirección. La sesión queda en `AWAITING_PAYMENT`.
3. En el panel de Artemis: **Ventas Online** → **Sandbox** → **pestaña Ventas**.
4. Abrí la venta (el `merchantOrderId` es el id del cobro, p. ej. `ord-20261021-f7d185c6`) y usá **Simular resultado**.

> Las opciones de simulación aparecen solo cuando la sesión está en `AWAITING_PAYMENT`. Antes de elegir método no se puede simular un pago.

Qué simular

Acción	Qué prueba
Pago exacto	El pago completo: la venta queda pagada y llega `checkout.payment.confirmed`
Otro monto	Un pago parcial: la venta queda en `CONFIRMING` y **no** se completa
Moneda incorrecta / otra cuenta / sin código	Pagos que no corresponden y se ignoran
Repetir el último pago	Un pago duplicado, que se ignora
Hacer que venza ahora	La expiración y `checkout.expired`
Hacer fallar el próximo intento	La preparación falla (antes de elegir método)
Dejar pendiente la conversión	Solo cripto: la liquidación queda `PENDING`

Tras un **pago exacto**, el panel muestra **"Pago recibido"** — eso confirma que el webhook llegó, se verificó la firma y se completó la compra.

Checklist antes de pasar a Live

Verificada el 22/10/2026 contra `pagarte.aurigital.com`:

- ☑ **Pago exacto** → la compra se completa y el panel muestra "Pago recibido"
- ☑ **Pago parcial** → PagArte marca PAGO PARCIAL y la compra queda `PENDING`, sin completarse (probado: ₡250 000 cobrados, 50 USDT pagados)
- ☑ **Evento duplicado** → el webhook responde `{"duplicate":true}` y no reprocesa (verificado con firmas Ed25519 reales)
- ☑ **Firma manipulada o ausente** → el servidor responde **400**
- ☑ **Persistencia** → los cobros y su estado sobreviven reinicios (`/api/health` → `"persistent": true`)

> El pago parcial es el caso que más importa: si la compra se completara con dinero de menos, en Live estarías entregando sin haber cobrado del todo. Ojo al probarlo: poner un monto **mayor** al cobrado es un sobrepago y sí completa la venta. Para probar el parcial, el monto debe ser **menor**.

Verificar que el embebido funciona

```
./scripts/check-pagarte-fix.sh
```

Crea una sesión real y revisa las cabeceras de `/checkout/{sessionId}`. Comprueba de paso que `STORE_ORIGIN` esté autorizado en el panel.

En Live sí es una dirección real

Con una llave `pgart_live_…` la dirección es de verdad y la wallet la acepta. Ahí no hay simulaciones: `/v1/checkout/test/…` no existe en Live, y cancelar una venta **no** reembolsa un pago que ya llegó.

Pasar a Live

Live se configura **aparte** de sandbox: otro widget, otras llaves y otra URL de notificaciones. Nada se hereda.

> **En Live el dinero es real.** Cancelar una venta **no** devuelve un pago que ya llegó, y no existen los endpoints de simulación (`/v1/checkout/test/…`).

---

1. Configurar el modo Live en el panel

En el panel de Artemis, salí de **Sandbox** y entrá a **Ventas Online** (sin el sufijo Sandbox).

1.1 Cómo recibís el dinero

Hacelo primero. Sin esto, crear una sesión devuelve `422 CHECKOUT_MERCHANT_NOT_READY`.

En **Ajustes** → **Liquidación de órdenes cripto**, decidí la conversión automática:

Conversión	Recibís	Cuándo conviene
Desactivada	El cripto directo (USDT) en tu wallet	Si querés quedarte en cripto
Activada	Colones al tipo de cambio del momento, menos comisiones	Si necesitás CRC

Con la conversión **desactivada**, Live solo ofrece las criptos para las que tengas wallet configurada. Configurá la wallet de USDT en Tron.

1.2 Activar pagos y autorizar el sitio

En **Configuración**:

1. **Recibir pagos:** activado.
2. **Sitios autorizados:** agregá `https://pagarte.aurigital.com`
3. Exacto, sin barra final. En Live solo se admite `https://`.
4. Este paso hay que repetirlo: los sitios de sandbox **no** pasan a Live.
5. **Métodos de pago:** marcá Cripto (y SINPE Móvil si registraste el teléfono).
6. **Guardá y copiá el nuevo ID del widget** — es distinto al de sandbox.

1.3 Crear la llave de API

En **Llaves de API**, creá una (por ejemplo "Produccion").

> 🔑 Se muestra **una sola vez**. Copiala al instante.

Debe empezar con `pgart_live_`.

1.4 Webhook

En **Notificaciones (webhook)**:

```
https://pagarte.aurigital.com/api/pagarte/webhook
```

Marcá los tres eventos: pagada, cancelada y vencida.

---

2. Actualizar Vercel

En **Settings** → **Environment Variables**, entorno **Production**, cambiá solo dos:

Variable	Valor nuevo
`PAGARTE_SECRET_KEY`	La llave `pgart_live_…`
`PAGARTE_WIDGET_ID`	El widget de Live

El resto (`PAGARTE_API_URL` , `STORE_ORIGIN` , `NEXT_PUBLIC_BASE_URL` , `NEXT_PUBLIC_PAGARTE_CHECKOUT_ORIGIN` , las de Redis) **no cambian**: la URL de la API es la misma y el modo lo decide el prefijo de la llave.

Después: **Redeploy**.

---

3. Verificar antes de cobrar

```
curl -s https://pagarte.aurigital.com/api/health | python3 -m json.tool
```

Esperado:

```
{
  "ok": true,
  "persistent": true,
  "storage": "KV_*",
  "mode": "LIVE",
  "storeOrigin": "https://pagarte.aurigital.com",
  "widgetConfigured": true
}
```

- `"mode": "LIVE"` → la llave de producción está activa.
- `"persistent": true` → Redis conectado. **Si dice** `false` **, parate acá:** sin

persistencia un pago confirmado se puede perder.

---

4. La primera prueba real

Empezá con un monto chico, no con todo el presupuesto.

1. Generá un cobro de **₡1 000** (menos de $2).
2. Pagalo desde tu propia wallet, con **USDT por Tron**: las comisiones de red son de centavos, a diferencia de Ethereum.
3. Confirmá que:
4. La dirección del checkout es **real** (empieza con `T`, 34 caracteres). Si vuelve a aparecer `PAGARTE_SANDBOX_…`, seguís en sandbox.
5. El panel muestra **"Pago recibido"**.
6. La venta figura **PAGADA** en el panel de Artemis.
7. La liquidación llega a tu wallet.

Recién después movés montos mayores.

> No es un gasto perdido: te estás pagando a vos mismo. Lo único que se pierde son las comisiones de red, que en Tron son centavos.

Qué cambia respecto a sandbox

	Sandbox	**Live**
Llave	`pgart_test_…`	`pgart_live_…`
Dirección cripto	Marcador `PAGARTE_SANDBOX_…`	Real, la wallet la acepta
Dinero	Simulado	**Real**
Simulaciones	Sí, desde el panel	**No existen**
Cancelar una venta	Sin consecuencias	**No reembolsa** un pago recibido

---

Volver a sandbox

Cambiá `PAGARTE_SECRET_KEY` y `PAGARTE_WIDGET_ID` por los de sandbox y redesplegá. El código detecta el modo por el prefijo de la llave, y el webhook valida `environment` en consecuencia, así que un evento de sandbox nunca puede completar una compra de Live ni al revés.

Persistencia de los cobros (Vercel KV / Redis)

El problema que resuelve

Los cobros vivían en un `Map` en memoria. En Vercel cada invocación puede caer en una instancia distinta, y las instancias se reciclan: un cobro creado por una instancia **desaparece** cuando esa instancia muere.

Eso rompía el caso más importante. El 21/10/2026 pasó en producción:

1. Se generó el cobro `ord-20261021-f7d185c6` → quedó en la memoria de una instancia.
2. Se simuló el pago en PagArte → la venta quedó **PAGADA** de su lado.
3. Llegó el webhook `checkout.payment.confirmed` … a otra instancia.
4. Esa instancia no conocía la compra → `unknown purchase` → evento descartado.
5. `/cobros` se veía vacío aunque el pago había ocurrido.

Un pago confirmado se perdió en silencio. Por eso la persistencia no es un lujo: sin ella el webhook no es confiable.

La solución

`lib/purchases.ts` guarda en Redis cuando está configurado, y cae a memoria solo para desarrollo local. Todas las funciones son `async`, así que el backend se puede cambiar sin tocar las rutas.

Clave	Qué guarda
`purchase:<id>`	El cobro completo
`purchases:index`	Sorted set por fecha, para listar del más nuevo al más viejo
`event:<eventId>`	Marca de evento procesado (TTL 30 días)

El TTL de los eventos evita que la base crezca sin límite; a los 30 días cualquier reintento de PagArte ya expiró.

Sigue haciendo falta aunque no haya lista de cobros

La lista se quitó del MVP, pero la persistencia **no** es opcional: el webhook busca la compra por `merchantOrderId`, y si no la encuentra descarta el evento. Sin Redis, un pago confirmado se pierde igual.

Configurar en Vercel

1. En el proyecto: **Storage** → **Create Database**.
2. Elegí **Upstash** (Serverless Redis) y su plan **Free**. > No elijas *Redis Cloud*: es otro proveedor del mismo marketplace y su > plan más barato cuesta $8/mes. Upstash alcanza de sobra para este volumen.

3. **Connect Project**, con Environments en *Production* y *Preview*, Custom Environment Variable Prefix en `KV` (Vercel sugiere `STORAGE`; cualquiera sirve, ver abajo) y *Sensitive* activado.
4. **Redeploy** para que el deploy activo tome las variables.

El prefijo no importa

El código acepta los tres que usa Vercel, así que no hay que acertarle:

Prefijo	De dónde viene
`KV_REST_API_*`	El convencional de Vercel
`STORAGE_REST_API_*`	El que sugiere el diálogo de conexión
`UPSTASH_REDIS_REST_*`	El propio de Upstash

`/api/health` informa cuál encontró, en el campo `storage`.

Verificar que quedó activo

```
curl -s https://pagarte.aurigital.com/api/health | python3 -m json.tool
```

`"persistent": true` significa que los cobros sobreviven reinicios. `false` significa que sigue en memoria y **se van a perder**.

En local

Sin las variables, usa el `Map` en memoria: suficiente para desarrollo, y se borra al reiniciar. Si querés probar la ruta de Redis en local, poné las mismas variables apuntando a tu base de Upstash.

Apéndice · Verificación en producción

Esta guía no es teórica: la integración que documenta está corriendo en `pagarte.aurigital.com` y procesó un cobro real.

El cobro de prueba en Live

Fecha	22 de octubre de 2026
Cobrado	₡1 000,00
Método	USDT · Tron (TRC20)
Pago recibido	2,23 USDT
Liquidado al comercio	2,222195 USDT
Estado	PAGADA · liquidación completada

El saldo llegó a la billetera de Artemis como **USDT (TRC20)**, sin conversión automática a colones.

Qué se verificó

Verificación	Entorno	Resultado
Crear sesión	Sandbox y Live	201
Idempotencia (misma clave → misma sesión)	Sandbox	200, mismo `sessionId`
Moneda distinta de CRC	Sandbox	422 `CHECKOUT_CURRENCY_NOT_ENABLED`
Origen no autorizado	Sandbox	403 `CHECKOUT_WIDGET_BLOCKED`
Checkout embebido	Sandbox y Live	Carga con su QR
Pago exacto	Sandbox y Live	Compra completada
Pago parcial	Sandbox	Queda `PENDING`, no se completa
Evento duplicado	Sandbox	`{"duplicate":true}`, no reprocesa
Firma inválida o ausente	Producción	400
Timestamp fuera de ventana (replay)	Local	400
`environment` cruzado (LIVE con llave TEST)	Local	Ignorado
Monto que no coincide	Local	Ignorado, compra intacta
Persistencia tras reinicio	Local y producción	Sobrevive

Un hallazgo que vale compartir

Durante la integración, un pago confirmado **se perdió en silencio**: el cobro vivía en memoria, Vercel recicló la instancia, y el webhook llegó a otra que no conocía la compra. PagArte la tenía como PAGADA; el sistema propio, no.

Es un error fácil de cometer con cualquier pasarela basada en webhooks, y la razón por la que este proyecto usa Redis. El detalle está en `PERSISTENCIA.md`.

Referencia rápida

```
API              https://api.artemis.money
SDK              https://checkout.pagarte.artemis.money/sdk/v1/checkout.js
Tipos (.d.ts)    https://checkout.pagarte.artemis.money/sdk/v1/checkout.d.ts
Llaves webhook   GET {API}/v1/checkout/webhook-keys
Documentación    https://pagarte.artemis.money/docs/es
```

Endpoint	Para qué
`POST /v1/checkout/sessions`	Crear la sesión
`GET /v1/checkout/sessions/{id}`	Estado y liquidación
`POST /v1/checkout/sessions/{id}/cancel`	Cancelar
`GET /v1/checkout/webhook-keys`	Llaves públicas para verificar firmas
`POST /v1/checkout/test/sessions/{id}/…`	Simulaciones (solo sandbox)

Código de referencia

La implementación completa es pública: **github.com/Aurigital/pagarte-checkout**

Archivo	Qué hace
`lib/pagarte.ts`	Cliente de la API
`lib/pagarte-webhook.ts`	Verificación Ed25519 y caché de llaves
`lib/purchases.ts`	Estado de compras (Redis) e idempotencia
`app/api/pagarte/webhook/route.ts`	Receptor con los seis chequeos
`components/PagArteCheckout.tsx`	Montaje del SDK